Robotics and Its Applications

Lab Assignment - 11

Name : Naishadh Rana

Enrollment Number : U23CS014

PART A: Simulation Environment

Task 1.1 Creating a Custom Gazebo World [25 Points] :

- First, create a ROS 2 workspace with packages:

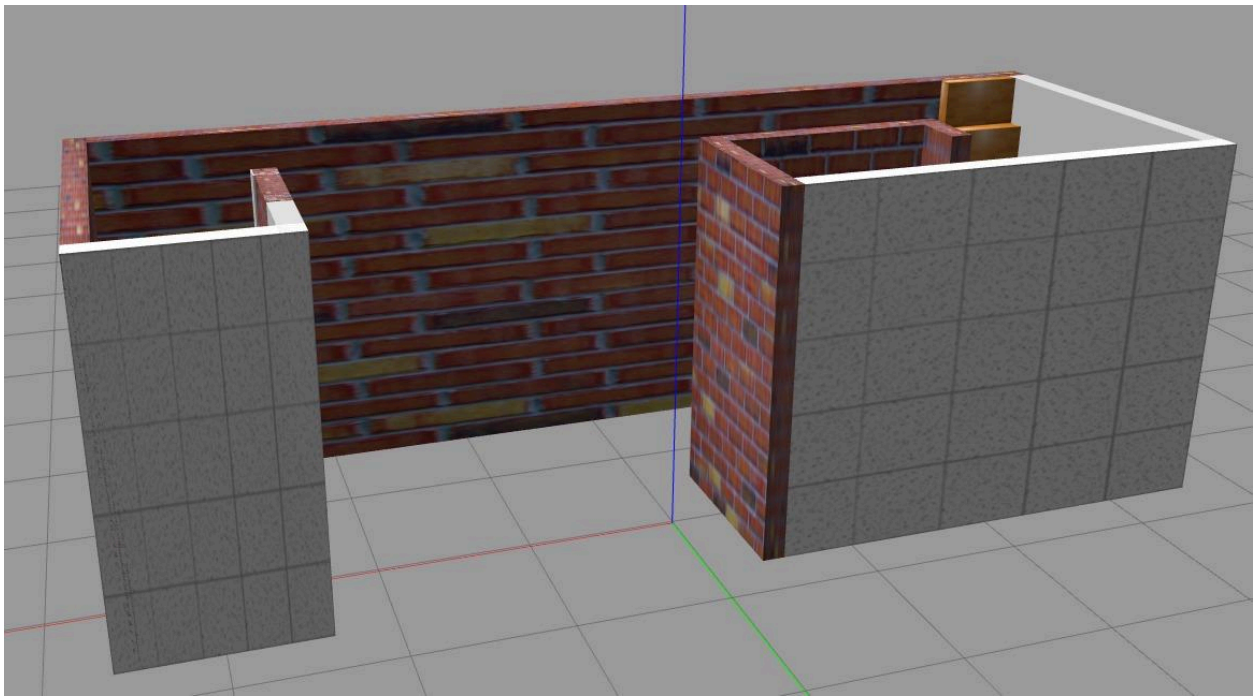
```
ros2 pkg create --build-type ament_python my_nav_bot
--dependencies rclpy gazebo_ros
```

1. Gazebo world:

```
<?xml version='1.0'?>
<sdf version='1.7'>
  <world name='Room'>
    <include>
      <uri>model://ground_plane</uri>
    </include>
    <include>
       <uri>model://Room</uri> <pose>0 0 0 0 0 0</pose>
     </include>
    <light name='sun' type='directional'>
      <cast_shadows>1</cast_shadows>
      <pose>0 0 10 0 -0 0</pose>
      <diffuse>0.8 0.8 0.8 1</diffuse>
```

```
      <specular>0.2 0.2 0.2 1</specular>
      <attenuation>
        <range>1000</range>
        <constant>0.9</constant>
        <linear>0.01</linear>
        <quadratic>0.001</quadratic>
      </attenuation>
      <direction>-0.5 0.1 -0.9</direction>
    </light>
    <scene>
      <shadows>false</shadows>
      <ambient>0.4 0.4 0.4 1</ambient>
      <background>0.7 0.7 0.7 1</background>
    </scene>
    <gui fullscreen='0'>
      <camera name='user_camera'>
        <pose frame=''>0.319654 -0.235002 9.29441 0 1.5138 0.009599</pose>
        <view_controller>orbit</view_controller>
        <projection_type>perspective</projection_type>
      </camera>
    </gui>
    <physics type="ode">
      <real_time_update_rate>1000.0</real_time_update_rate>
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1</real_time_factor>
      <ode>
        <solver>
          <type>quick</type>
          <iters>150</iters>
          <precon_iters>0</precon_iters>
          <sor>1.400000</sor>
          <use_dynamic_moi_rescaling>1</use_dynamic_moi_rescaling>
        </solver>
        <constraints>
          <cfm>0.00001</cfm>
          <erp>0.2</erp>
          <contact_max_correcting_vel>2000.000000</contact_max_correcting_vel>
          <contact_surface_layer>0.01000</contact_surface_layer>
        </constraints>
```

```
      </ode>
    </physics>
    <gravity>0 0 -9.81</gravity>
    <magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
    <atmosphere type='adiabatic' />
  </world>
</sdf>
```



2. Robot Model (Xacro):

```
<?xml version="1.0"?>
<robot name="differential_drive_robot"
xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:property name="body_link_x_dim" value="1"/>
  <xacro:property name="body_link_y_dim" value="0.6"/>
  <xacro:property name="body_link_z_dim" value="0.3"/>
  <xacro:property name="wheel_link_radius" value="0.15"/>
  <xacro:property name="wheel_link_length" value="0.1"/>
  <xacro:property name="wheel_link_z_location" value="-0.1"/>
  <xacro:property name="caster_link_radius" value="0.15"/>
```

```
  <xacro:property name="caster_link_z_location" value="-0.1"/>
  <xacro:property name="body_density" value="2710.0"/>
  <xacro:property name="wheel_density" value="2710.0"/>
  <xacro:property name="caster_density" value="2710.0"/>
  <xacro:property name="pi_constant" value="3.14159265"/>
  <xacro:property name="body_mass"
value="${body_density*body_link_x_dim*body_link_y_dim*body_link_z_dim}"/>
  <xacro:property name="wheel_mass"
value="${wheel_density*wheel_link_radius*wheel_link_radius*pi_constant*wheel_link
_length}"/>
  <xacro:property name="caster_mass"
value="${caster_density*(4.0/3.0)*pi_constant*caster_link_radius*caster_link_radi
us*caster_link_radius}"/>
  <xacro:property name="Iz_wheel"
value="${wheel_mass*wheel_link_radius*wheel_link_radius}"/>
  <xacro:property name="I_wheel"
value="${(1.0/12.0)*wheel_mass*(3.0*wheel_link_radius*wheel_link_radius+wheel_lin
k_length*wheel_link_length)}"/>
  <xacro:property name="I_caster"
value="${(2.0/5.0)*caster_mass*caster_link_radius*caster_link_radius}"/>
  <xacro:macro name="inertia_wheel">
    <inertial>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <mass value="${wheel_mass}"/>
    <inertia ixx="${I_wheel}" ixy="0.0" ixz="0.0" iyy="${I_wheel}" iyz="0"
izz="${Iz_wheel}"/>
    </inertial>
  </xacro:macro>
  <xacro:macro name="inertia_caster">
    <inertial>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <mass value="${caster_mass}"/>
    <inertia ixx="${I_caster}" ixy="0.0" ixz="0.0" iyy="${I_caster}" iyz="0"
izz="${I_caster}"/>
    </inertial>
  </xacro:macro>

  <xacro:include filename="$(find my_nav_bot)/model/robot.gazebo" />
```

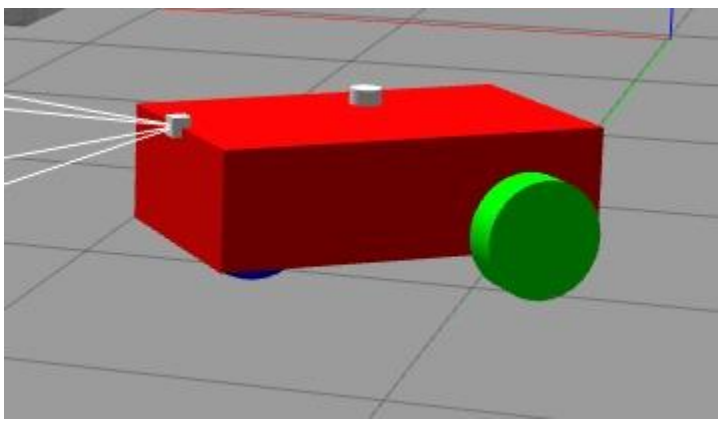
```
  <link name="dummy">
  </link>
  <joint name="dummy_joint" type="fixed">
    <parent link="dummy"/>
    <child link="body_link" />
  </joint>
  <link name="body_link">
    <visual>
    <geometry>
        <box size="${body_link_x_dim} ${body_link_y_dim} ${body_link_z_dim}" />
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <material name="red">
        <color rgba="1 0 0 1" />
    </material>
    </visual>
    <collision>
    <geometry>
        <box size="${body_link_x_dim} ${body_link_y_dim} ${body_link_z_dim}" />
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    </collision>
    <inertial>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <mass value="${body_mass}"/>
    <inertia
ixx="${(1/12)*body_mass*(body_link_y_dim*body_link_y_dim+body_link_z_dim*body_lin
k_z_dim)}" ixy="0" ixz="0"

iyy="${(1/12)*body_mass*(body_link_x_dim*body_link_x_dim+body_link_z_dim*body_lin
k_z_dim)}" iyz="0"

izz="${(1/12)*body_mass*(body_link_y_dim*body_link_y_dim+body_link_x_dim*body_lin
k_x_dim)}" />
    </inertial>
  </link>
  <joint name="wheell_joint" type="continuous">
    <parent link="body_link"/>
    <child link="wheell_link" />
```

```
    <origin xyz="${-2*wheel_link_radius}
${-body_link_y_dim/2-wheel_link_length/2} ${wheel_link_z_location}" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
    <limit effort="1000" velocity="1000"/>
    <dynamics damping="1.0" friction="1.0"/>
  </joint>
  <link name="wheell_link">
    <visual>
    <origin rpy="1.570795 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="${wheel_link_length}" radius="${wheel_link_radius}" />
    </geometry>
    <material name="green">
        <color rgba="0 1 0 1" />
    </material>
    </visual>
    <collision>
    <origin rpy="1.570795 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="${wheel_link_length}" radius="${wheel_link_radius}" />
    </geometry>
    </collision>
    <xacro:inertia_wheel />
  </link>
  <joint name="wheel2_joint" type="continuous">
    <parent link="body_link"/>
    <child link="wheel2_link" />
    <origin xyz="${-2*wheel_link_radius} ${body_link_y_dim/2+wheel_link_length/2}
${wheel_link_z_location}" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
    <limit effort="1000" velocity="1000"/>
    <dynamics damping="1.0" friction="1.0"/>
  </joint>
  <link name="wheel2_link">
    <visual>
    <origin rpy="1.570795 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="${wheel_link_length}" radius="${wheel_link_radius}" />
    </geometry>
```

```
    <material name="green"/>
    </visual>
    <collision>
    <origin rpy="1.570795 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="${wheel_link_length}" radius="${wheel_link_radius}" />
    </geometry>
    </collision>
    <xacro:inertia_wheel />
  </link>
  <joint name="caster_joint" type="fixed">
    <parent link="body_link"/>
    <child link="caster_link" />
    <origin xyz="${2*caster_link_radius} 0 ${caster_link_z_location}" rpy="0 0
0"/>
  </joint>
  <link name="caster_link">
    <visual>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <sphere radius="${caster_link_radius}" />
    </geometry>
    <material name="blue">
        <color rgba="0 0 1 1" />
    </material>
    </visual>
    <collision>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <sphere radius="${caster_link_radius}" />
    </geometry>
    </collision>
    <xacro:inertia_caster />
  </link>
  <link name="lidar_link">
    <visual>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="0.04" radius="0.04" />
```

```
    </geometry>
    <material name="lidar_material">
        <color rgba="0.0 0.0 0.0 1.0"/>
    </material>
    </visual>
    <collision>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <cylinder length="0.04" radius="0.04" />
    </geometry>
    </collision>
    <inertial>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <mass value="0.1"/>
    <inertia ixx="0.000001" ixy="0" ixz="0" iyy="0.0000001" iyz="0"
izz="0.0000001"/>
    </inertial>
  </link>
  <joint name="lidar_joint" type="fixed">
    <parent link="body_link" />
    <child link="lidar_link" />
    <origin xyz="0 0 ${body_link_z_dim/2 + 0.05}" rpy="0 0 0"/>
  </joint>
  <link name="camera_link">
    <visual>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <box size="0.05 0.05 0.05"/>
    </geometry>
    <material name="camera_material">
        <color rgba="0.0 0.0 0.0 1.0"/>
    </material>
    </visual>
    <collision>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <geometry>
        <box size="0.05 0.05 0.05"/>
    </geometry>
    </collision>
```

```
    <inertial>
    <origin rpy="0 0 0" xyz="0 0 0"/>
    <mass value="0.1"/>
    <inertia ixx="0.000001" ixy="0" ixz="0" iyy="0.0000001" iyz="0"
izz="0.0000001"/>
    </inertial>
  </link>
  <joint name="camera_joint" type="fixed">
    <parent link="body_link"/>
    <child link="camera_link"/>
    <origin xyz="${body_link_x_dim/2} 0 ${body_link_z_dim/2}" rpy="0 0 0"/>
  </joint>
  </robot>
```

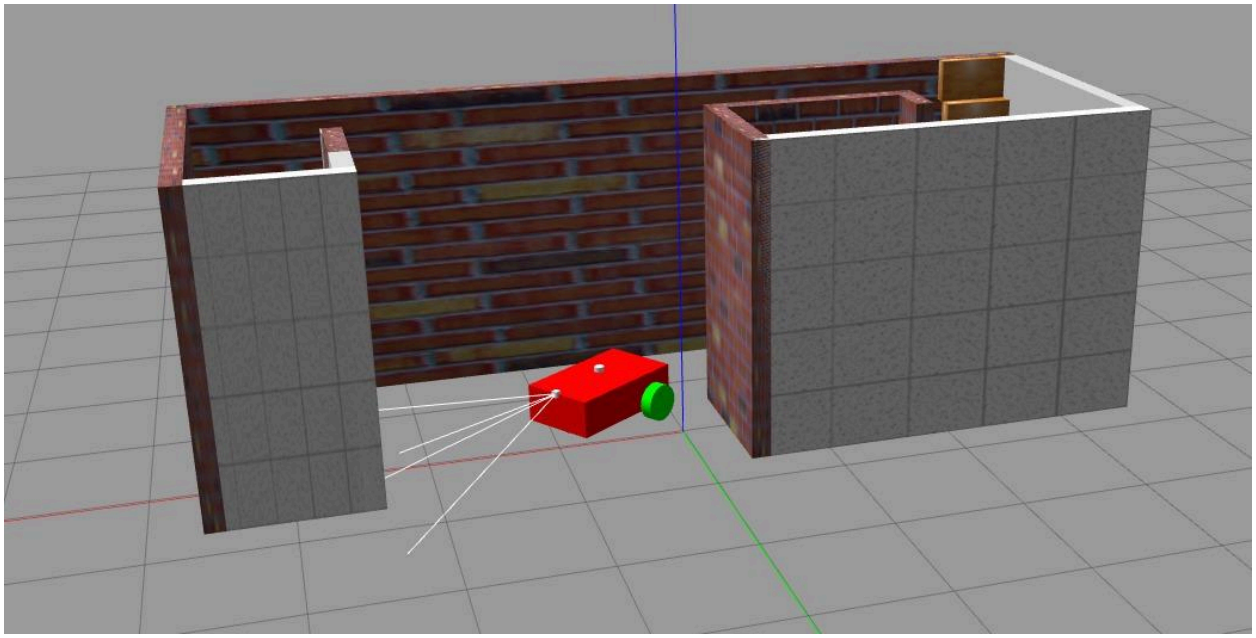


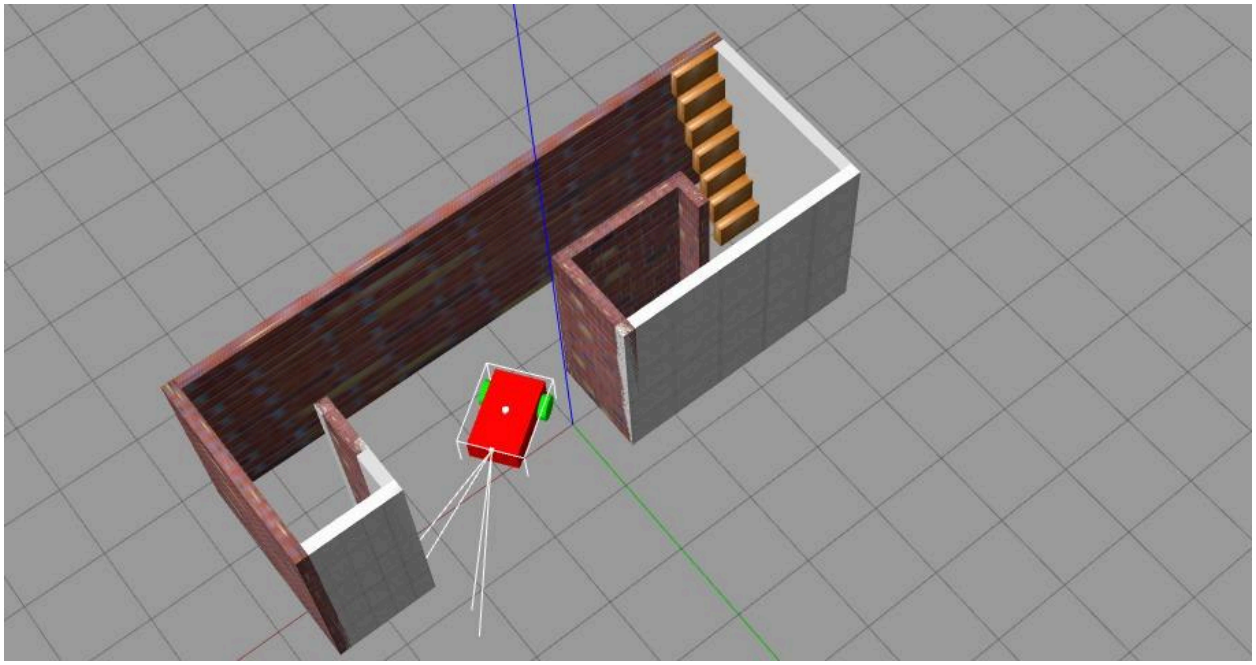
3. Robot.gazebo file:

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <gazebo>
    <plugin name="differential_drive_controller"
filename="libgazebo_ros_diff_drive.so">
      <ros>
        <namespace>/</namespace>
      </ros>

      <left_joint>wheell_joint</left_joint>
      <right_joint>wheel2_joint</right_joint>

      <wheel_separation>${body_link_y_dim + wheel_link_length}</wheel_separation>
      <wheel_diameter>${2 * wheel_link_radius}</wheel_diameter>

      <max_wheel_torque>1000</max_wheel_torque>
      <max_wheel_acceleration>1.0</max_wheel_acceleration>

      <odometry_frame>odom</odometry_frame>
      <robot_base_frame>dummy</robot_base_frame>
      <publish_odometry>true</publish_odometry>
      <publish_odometry_tf>true</publish_odometry_tf>
      <publish_wheel_tf>false</publish_wheel_tf>
      <update_rate>100</update_rate>

      <command_topic>cmd_vel</command_topic>
      <odometry_topic>odom</odometry_topic>
    </plugin>
  </gazebo>

  <gazebo reference="lidar_link">
    <sensor type="ray" name="lidar_sensor">
      <pose>0 0 0 0 0 0</pose>
```

```
      <visualize>true</visualize>
      <update_rate>10</update_rate>
      <ray>
        <scan>
          <horizontal>
            <samples>720</samples>
            <resolution>1</resolution>
            <min_angle>-3.14159</min_angle>
            <max_angle>3.14159</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.1</min>
          <max>12.0</max>
          <resolution>0.01</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </ray>
      <plugin name="gazebo_ros_lidar_plugin"
filename="libgazebo_ros_ray_sensor.so">
        <ros>
          <namespace>/</namespace>
          <output_type>sensor_msgs/LaserScan</output_type>
          <topic_name>scan</topic_name>
        </ros>
        <frame_name>lidar_link</frame_name>
      </plugin>
    </sensor>
  </gazebo>

  <gazebo reference="camera_link">
    <sensor type="camera" name="camera_sensor">
      <update_rate>30.0</update_rate>
      <camera name="head">
        <horizontal_fov>1.396263</horizontal_fov>
```

```
        <image>
          <width>800</width>
          <height>800</height>
          <format>R8G8B8</format>
        </image>
        <clip>
          <near>0.02</near>
          <far>300</far>
        </clip>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.007</stddev>
        </noise>
      </camera>
      <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
        <ros>
          <namespace>/</namespace>
          <topic_name>camera/image_raw</topic_name>
          <camera_info_topic_name>camera/camera_info</camera_info_topic_name>
          <frame_name>camera_link</frame_name>
        </ros>
      </plugin>
    </sensor>
  </gazebo>

</robot>
```





PART B: Autonomous Navigation using SLAM

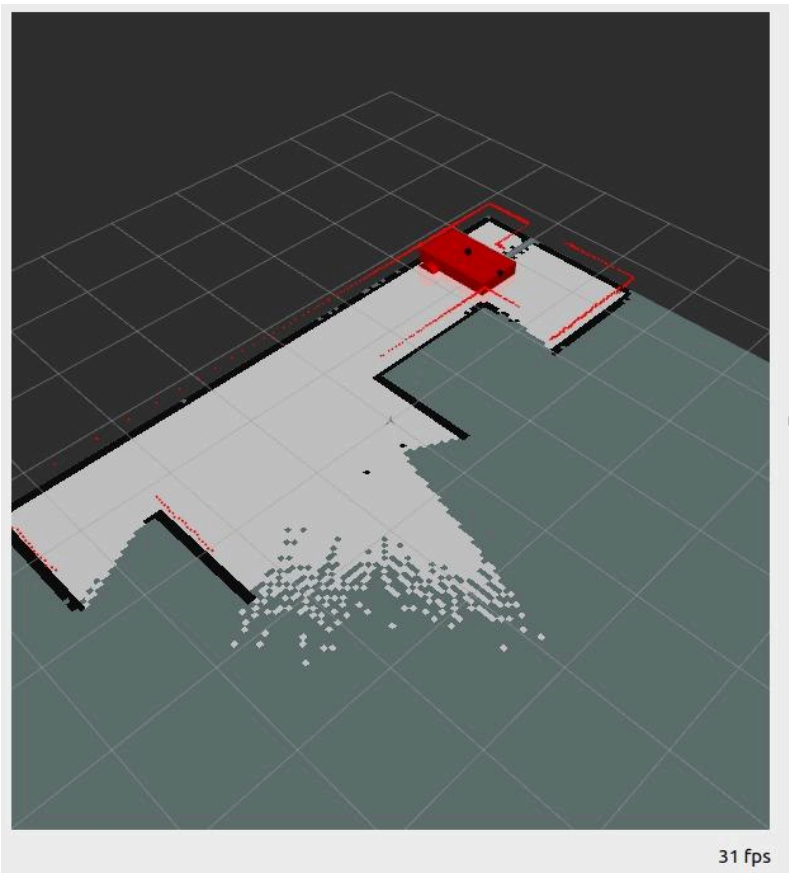
Task 2.1: Mapping Using SLAM &
Task 2.2: Autonomous Navigation and Obstacle Avoidance

1. bringup_slam.launch.py

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    package_name = 'my_nav_bot'

    # Paths
    pkg_share = get_package_share_directory(package_name)
    world_file = os.path.join(pkg_share, 'model', 'room.world')
    xacro_file = os.path.join(pkg_share, 'model', 'robot.xacro')
    rviz_config_file = os.path.join(pkg_share, 'config', 'rviz_slam_config.rviz')
    slam_params_file = os.path.join(pkg_share, 'config',
'mapper_params_online_async.yaml')

    # Process Xacro
    robot_description_raw = xacro.process_file(xacro_file).toxml()

    # Gazebo
    gazebo_ros_launch = PythonLaunchDescriptionSource(
        os.path.join(get_package_share_directory('gazebo_ros'), 'launch',
'gazebo.launch.py')
    )
    gazebo_launch = IncludeLaunchDescription(
        gazebo_ros_launch,
        launch_arguments={'world': world_file, 'verbose': 'false'}.items()
    )
```

```
    # Spawn Robot
    spawn_model_node = Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=['-topic', 'robot_description', '-entity',
'differential_drive_robot',
                   '-x', '0.0', '-y', '0.0', '-z', '0.2', '-R', '0.0', '-P',
'0.0', '-Y', '0.0'],
        output='screen'
    )

    # Robot State Publisher
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_description_raw, 'use_sim_time':
True}]
    )

    # SLAM Toolbox
    slam_toolbox_node = Node(
        package='slam_toolbox',
        executable='async_slam_toolbox_node',
        name='slam_toolbox',
        output='screen',
        parameters=[slam_params_file, {'use_sim_time': True}],
        remappings=[('scan', '/scan')]
    )

    # RViz
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_config_file],
        parameters=[{'use_sim_time': True}]
```

```
    )

    # Launch Description
    ld = LaunchDescription()
    ld.add_action(gazebo_launch)
    ld.add_action(spawn_model_node)
    ld.add_action(robot_state_publisher_node)
    ld.add_action(slam_toolbox_node)
    ld.add_action(rviz_node)

    return ld
```



PART C: Visualization and Analysis

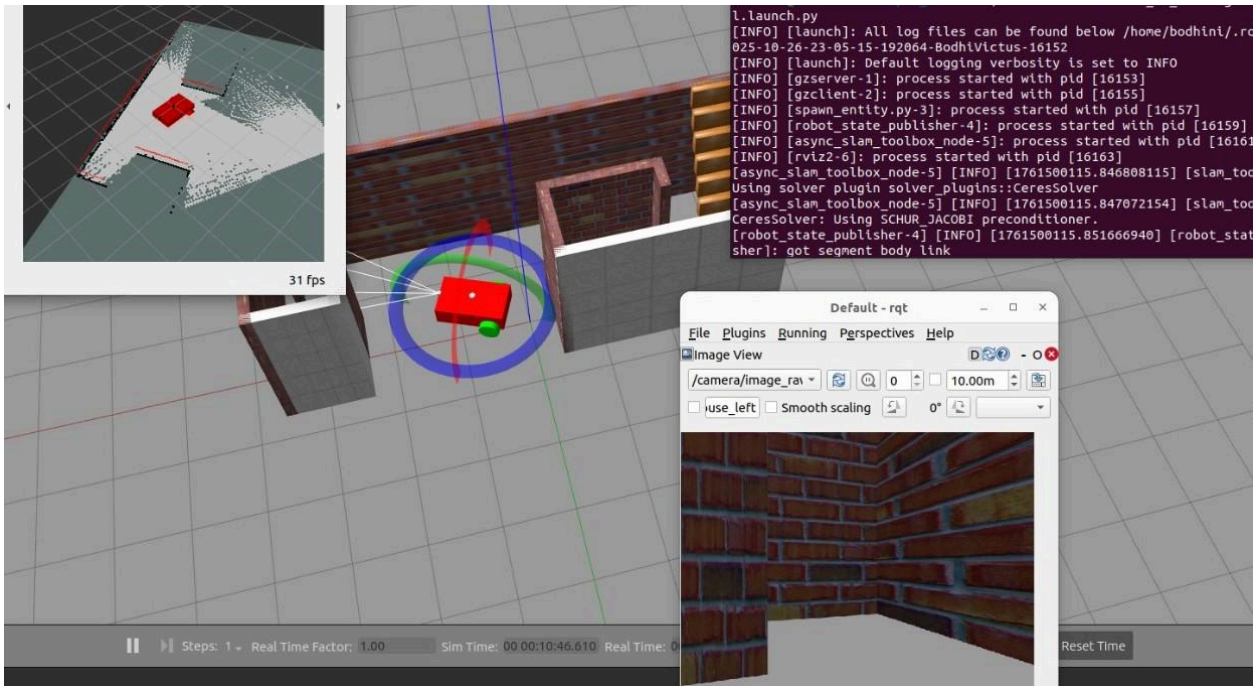
Task 3.1: Visualization using RViz2

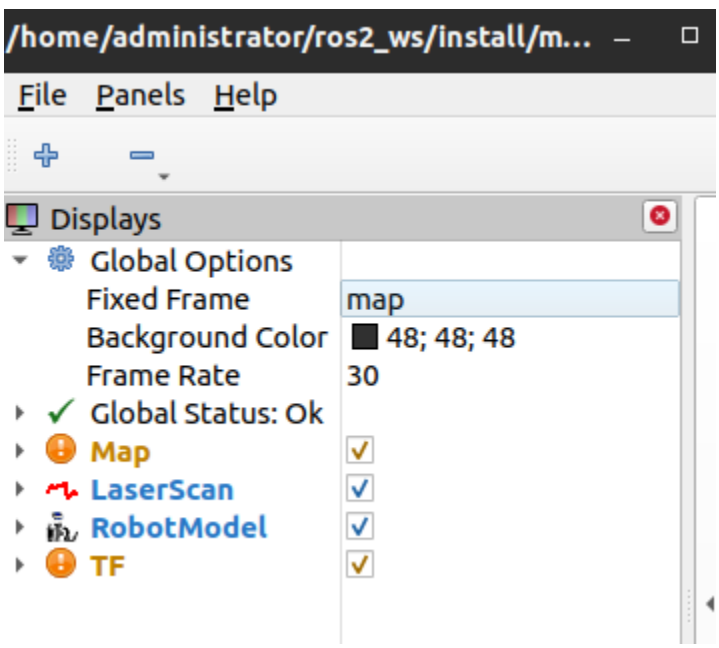
bringup_nav.launch.py:

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():

    package_name = 'my_nav_bot'
    pkg_share = get_package_share_directory(package_name)

    # Paths to config files
    nav2_params_file = os.path.join(pkg_share, 'config', 'nav2_params.yaml')
    map_file = os.path.join(pkg_share, 'maps', 'my_map.yaml')
    rviz_config_file = os.path.join(pkg_share, 'config', 'nav_config.rviz')
    world_file = os.path.join(pkg_share, 'model', 'room.world')
    xacro_file = os.path.join(pkg_share, 'model', 'robot.xacro')

    # Launch arguments
    use_sim_time = LaunchConfiguration('use_sim_time', default='true')
    autostart = LaunchConfiguration('autostart', default='true')

    # Process Xacro
    robot_description_raw = xacro.process_file(xacro_file).toxml()

    # Gazebo
    gazebo_ros_launch = PythonLaunchDescriptionSource(
        os.path.join(get_package_share_directory('gazebo_ros'), 'launch',
'gazebo.launch.py')
    )
    gazebo_launch = IncludeLaunchDescription(
```

```
        gazebo_ros_launch,
        launch_arguments={'world': world_file, 'verbose': 'false'}.items()
    )

    # Spawn Robot
    spawn_model_node = Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=['-topic', 'robot_description', '-entity',
'differential_drive_robot',
                   '-x', '0.0', '-y', '0.0', '-z', '0.2', '-R', '0.0', '-P',
'0.0', '-Y', '0.0'],
        output='screen'
    )

    # Robot State Publisher
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_description_raw, 'use_sim_time':
True}]
    )

    # Nav2 Bringup
    nav2_bringup_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(get_package_share_directory('nav2_bringup'), 'launch',
'bringup_launch.py')
        ),
        launch_arguments={
            'use_sim_time': use_sim_time,
            'params_file': nav2_params_file,
            'map': map_file,
            'autostart': autostart
        }.items()
    )

    # RViz
```

```python
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_config_file],
        parameters=[{'use_sim_time': use_sim_time}]
    )

    # Launch Description
    ld = LaunchDescription()

    # Add launch arguments
    ld.add_action(DeclareLaunchArgument('use_sim_time', default_value='true'))
    ld.add_action(DeclareLaunchArgument('autostart', default_value='true'))

    # Add nodes
    ld.add_action(gazebo_launch)
    ld.add_action(spawn_model_node)
    ld.add_action(robot_state_publisher_node)
    ld.add_action(nav2_bringup_launch)
    ld.add_action(rviz_node)

    return ld
```


31 fps
l.launch.py
[INFO] [launch]: All log files can be found below /home/bodhini/.ro
025-10-26-23-05-15-192064-BodhiVictus-16152
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [gzserver-1]: process started with pid [16153]
[INFO] [gzclient-2]: process started with pid [16155]
[INFO] [spawn_entity.py-3]: process started with pid [16157]
[INFO] [robot_state_publisher-4]: process started with pid [16159]
[INFO] [async_slam_toolbox_node-5]: process started with pid [16161
[INFO] [rviz2-6]: process started with pid [16163]
[async_slam_toolbox_node-5] [INFO] [1761500115.846808115] [slam_too
Using solver plugin solver_plugins::CeresSolver
[async_slam_toolbox_node-5] [INFO] [1761500115.847072154] [slam_too
CeresSolver: Using SCHUR_JACOBI preconditioner.
[robot_state_publisher-4] [INFO] [1761500115.851666940] [robot_stat
sher]: got segment body link
Default - rqt
File Plugins Running Perspectives Help
Image View
/camera/image_rav
10.00m
use_left
Smooth scaling
0°
Steps: 1
Real Time Factor: 1.00
Sim Time: 00 00:10:46.610
Real Time:
Reset Time


/home/administrator/ros2_ws/install/m...
File Panels Help
Displays
Global Options
Fixed Frame
map
Background Color
48; 48; 48
Frame Rate
30
Global Status: Ok
Map
LaserScan
RobotModel
TF

PART D: Exploratory Discussion

Task 4.1: Conceptual Analysis and Reflection (5 Points)
Provide a concise written summary demonstrating your understanding of the underlying principles. Explain the working principle of SLAM, describing how it solves the chicken-and-egg problem of simultaneous mapping and localization. Detail the roles of the key components within the ROS 2 Navigation stack, such as the AMCL node for localization, the planner_server for global path planning, and the controller_server for local motion control and trajectory following. Finally, describe how reactive obstacle avoidance is achieved using the local costmap and the local planner. Conclude this section by reflecting on the main challenges you encountered during the lab and how you resolved them.

SLAM stands for **Simultaneous Localization and Mapping**. Its main idea is that a robot builds a map of an unknown environment while also figuring out its own position inside that same map at the same time. This creates a chicken-and-egg problem, because the robot needs a map to know where it is, but it also needs to know where it is in order to build the map correctly. SLAM solves this by continuously using sensor data such as laser scans or camera readings, comparing them with previous observations, and updating both the map and the robot's estimated position step by step. In simple words, the robot keeps correcting itself while creating the map.

In the ROS 2 Navigation stack, each key component has a specific role. The **AMCL node** is used for localization. It helps the robot estimate its position on a known map using sensor data and particle filtering.

The **planner_server** is responsible for global path planning. It calculates a path from the robot's current position to the goal while avoiding large obstacles on the map.

The **controller_server** handles local motion control and trajectory following. It takes the global path and generates smooth movement commands so the robot can actually follow the path in real time.

Reactive obstacle avoidance is achieved using the **local costmap** and the **local planner**. The local costmap contains nearby obstacles detected by the robot's sensors, so it changes dynamically as the

environment changes. The local planner uses this information to adjust the robot’s movement and avoid collisions while moving toward the goal. This is useful because even if the global path is correct, unexpected obstacles can appear in front of the robot, and the local planner helps it react immediately.

The main challenges I faced during the lab were getting the navigation system configured correctly and understanding how all the nodes work together. At first, it was difficult to see the difference between global planning, localization, and local control. I resolved this by checking the data flow between the nodes and testing each part separately.